

VoyagerAI — Autonomous Multi-Agent AI Travel Planner

Week 2 → Week 8 Execution Roadmap

0. Reality Check (read this first)

Your original spec lists 8 agents, RAG, full auth, admin dashboard, analytics, monitoring, notifications, dark mode, packing lists, expense breakdown, maps, and cloud deployment — all in 7 weeks, with 4 people who also have other coursework. That's a 3-4 month scope compressed into 7 weeks.

To actually **win** rather than half-finish everything, this roadmap splits features into:

- **MVP (must work for demo)** — built weeks 2-6, hardened weeks 7-8
- **Stretch (build only if ahead of schedule)** — clearly marked each week

If you're behind in any week, cut stretch items first. A judge remembers a smooth 5-agent demo far more than a broken 8-agent one.

1. Tech Stack & Why

Layer	Choice	Why
Backend framework	FastAPI (Python 3.11+)	Async native, auto OpenAPI docs, best ecosystem fit for LangChain/LangGraph
Agent orchestration	LangGraph	Purpose-built for stateful multi-agent graphs with conditional routing — exactly your Coordinator/Planner/sub-agent pattern
Tool calling / RAG glue	LangChain	Mature tool-calling abstractions, works natively with LangGraph
LLM	Gemini 2.0 Flash (primary) via Google AI Studio	Free tier generous enough for a student project's demo load; fast; strong tool-calling. Keep an OpenAI key as a documented fallback in <code>.env.example</code>

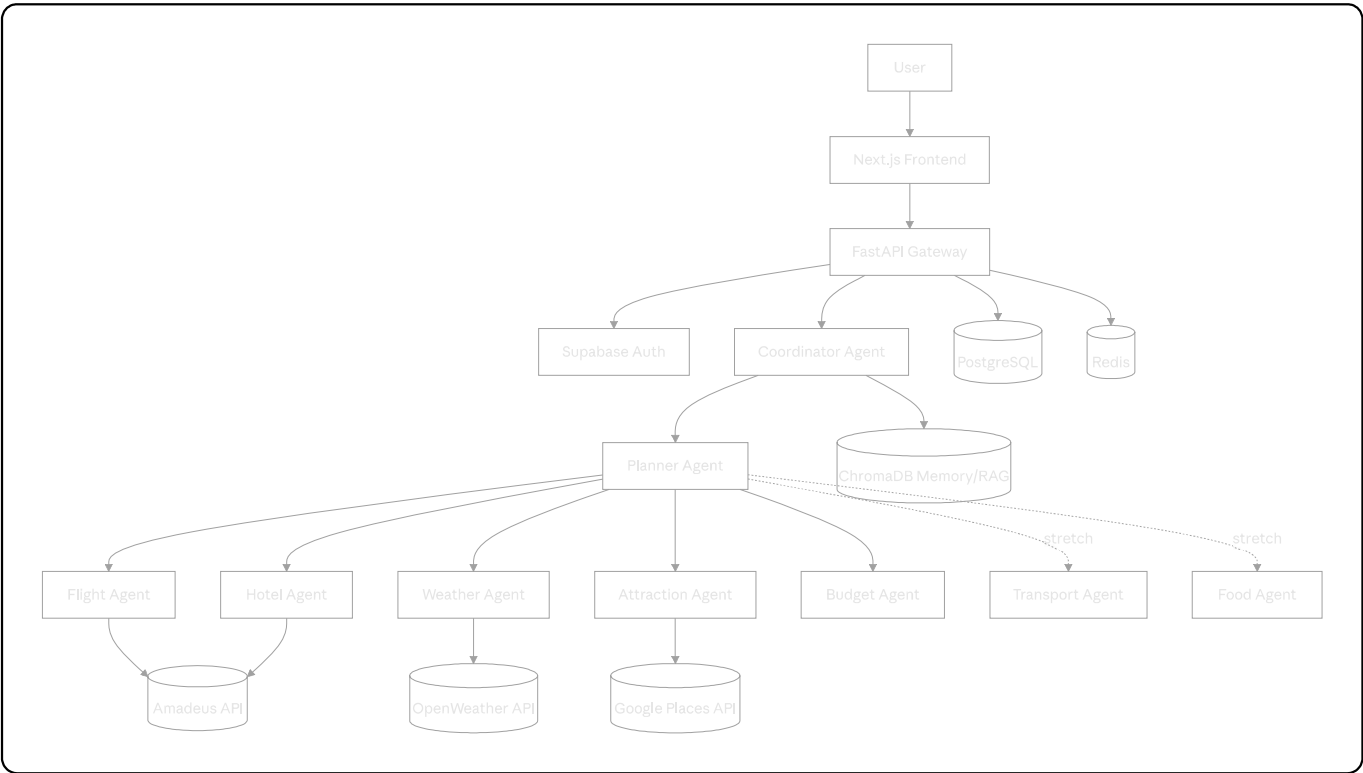
Layer	Choice	Why
Vector DB	ChromaDB (local, embedded)	Zero infra cost, runs in-process, perfect for conversation memory + RAG on destination knowledge
Relational DB	PostgreSQL via Supabase	Free tier, gives you DB + Auth + Storage in one place — cuts a full week of infra work
Cache / session	Redis (Upstash free tier)	Rate limiting, session state, agent-run caching
Frontend	Next.js 14 (App Router) + Tailwind CSS + shadcn/ui	Fast to build polished UI, SSR for good demo performance, huge component ecosystem
Auth	Supabase Auth	Pre-built, secure, saves a week vs rolling your own JWT flow
Maps/Places	Google Places API + Maps JS API	Attraction data + map rendering in one vendor
Weather	OpenWeather API	Free tier, simple REST
Flights/Hotels	Amadeus Self-Service API (sandbox)	Free sandbox with real flight/hotel search — much more credible than mock data for judges
Containerization	Docker + docker-compose	Local parity, required for any real deployment story
CI/CD	GitHub Actions	Free, standard, easy to demo "green checkmarks"
Deployment	Frontend → Vercel, Backend → Render (or Railway), DB → Supabase	All have generous free tiers, all deploy from GitHub in minutes
Monitoring	Sentry (free tier) + structured logging	Real error tracking without infra overhead

2. Team Roles

Member	Primary Focus	Also owns
A	Backend + external API integrations	Auth wiring, API docs
B	Frontend + UI/UX	Dashboard, chat interface, responsive design
C	AI + Agents (LangGraph)	Prompt engineering, RAG, conflict resolution logic
D	Database + DevOps + Testing	Schema, migrations, Docker, CI/CD, test coverage

All four attend a 15-min daily standup and a 1-hr Friday integration session where everyone's branch gets merged and demoed together — this is what prevents "week 7 integration hell."

3. System Architecture (high level)



Agent scope decision: Coordinator, Planner, Flight, Hotel, Weather, Attraction, Budget = **MVP (7 agents)**. Transport and Food agents are **stretch** — if built, they can be simple single-tool agents reusing the Attraction agent's pattern, not full new subsystems.

4. Target Folder Structure (built incrementally, not all at once)

```
voyagerai/
├─ backend/
│   └─ app/
│       ├── api/v1/           # FastAPI routers
│       ├── agents/          # LangGraph agent definitions
│       ├── core/            # config, security, logging
│       ├── db/              # SQLAlchemy models, session
│       ├── migrations/      # Alembic
│       ├── repositories/    # repository pattern (DB access)
│       ├── schemas/         # Pydantic request/response models
│       ├── services/        # business logic layer
│       ├── tools/           # external API wrapper functions
│       ├── rag/             # ChromaDB setup, embeddings
│       └─ main.py
│   └─ tests/
├─ Dockerfile
└─ requirements.txt
├─ frontend/
│   ├── app/                 # Next.js App Router pages
│   ├── components/
│   ├── lib/
│   ├── hooks/
│   └─ Dockerfile
├─ docs/
│   ├── architecture/       # diagrams: sequence, ER, DFD
│   └─ api/
├─ .github/workflows/
├─ docker-compose.yml
└─ README.md
```

5. Week-by-Week Roadmap

WEEK 2 — Foundations & Architecture Lock-in

(due today — this is the "we have a real plan" submission, not working code yet)

Objectives

MVP Features this week: none yet — this is scaffolding **Stretch:** none

Folder structure: create the full skeleton above with empty (`__init__.py`)/placeholder files

Tasks

- **A:** Init FastAPI project, health-check endpoint (`GET /api/v1/health`), (`.env.example`), connect to Supabase Postgres
- **B:** Init Next.js project, Tailwind + shadcn/ui setup, call the health-check endpoint and render status on homepage
- **C:** Draft the LangGraph agent architecture diagram (which agents, what state flows between them), pick Gemini API, get a key, run one "hello world" LangGraph node calling Gemini
- **D:** Design ER diagram (Users, Trips, Conversations, Itineraries, AgentRuns tables), write Alembic migration for it, set up Docker + docker-compose for local Postgres/Redis, init GitHub repo + branch protection + Actions skeleton (lint + test job, even if tests are empty)

Deliverable (demoable): Frontend page that pings backend, backend pings DB successfully, one LangGraph node returns a real Gemini response in a script — plus the architecture diagrams.

Git workflow: (`main`) protected; feature branches (`feat/backend-init`), (`feat/frontend-init`), (`feat/agent-skeleton`), (`feat/db-schema`); PR + 1 review required to merge into (`develop`); merge to (`main`) only at week-end.

Testing: Set up pytest + Jest scaffolding (no real tests yet, just CI running green on empty suites).

Documentation: SRS finalized doc, ER diagram, architecture diagram (mermaid above, refined), (`README.md`) with setup instructions.

Submission checklist: SRS ☒ | Architecture diagram ☒ | ER diagram ☒ | Repo with skeleton ☒ | Tech stack justification doc ☒

Demo checklist: Show repo structure, show CI passing, show FE→BE→DB ping working live.

Risks / buffer: API key approval delays (Amadeus sandbox signup can take a day) → apply for all API keys (Gemini, OpenWeather, Google Places, Amadeus) *today*, not when you need them in week 4.

WEEK 3 — Auth, Data Layer & Basic Chat Shell

Objectives

- Real user accounts working end to end
- Chat UI that talks to a stub AI endpoint
- Coordinator Agent skeleton running as a real LangGraph graph

MVP Features: Sign up / login / logout, start a new "trip conversation," persist messages to DB
Stretch: none yet

Folder structure additions: `app/api/v1/auth.py`, `app/api/v1/trips.py`,
`app/repositories/`, `frontend/app/(auth)/`, `frontend/app/chat/`

Tasks

- **A:** Wire Supabase Auth into FastAPI (JWT verification middleware), `POST /trips`, `GET /trips/{id}/messages`, `POST /trips/{id}/messages` (stub AI reply for now)
- **B:** Build login/signup pages, chat UI shell (message list + input box, streaming-ready), trip creation flow
- **C:** Build Coordinator Agent as a real LangGraph `StateGraph` with one node (echoes back a structured "planning started" response); define the shared agent state schema (destination, dates, budget, preferences, agent outputs)
- **D:** Repository pattern for `User`, `Trip`, `Message` models; Redis session caching; write first real unit tests for repositories; CI now runs actual tests

Deliverable: User can sign up, log in, start a trip, type a message, and see a real (if basic) response persisted and reloaded on refresh.

Git workflow: Same branching model; require passing CI before merge from this week on.

Testing: Unit tests for auth middleware, repository CRUD, and the Coordinator Agent's state transitions (>50% coverage on backend by week end).

Documentation: API docs auto-generated via FastAPI's OpenAPI, update README with auth setup steps.

Submission checklist: Working auth ☒ | Chat persists to DB ☒ | Coordinator Agent skeleton ☒ | Tests passing in CI ☒

Demo checklist: Live signup → login → create trip → send message → refresh page → message still there.

Risks / buffer: Supabase Auth + FastAPI JWT verification has a learning curve — Member A should timebox this to 1 day and pull in Member D if stuck.

Est. hours: ~14-16 hrs each.

WEEK 4 — Real Multi-Agent Intelligence (Core)

Objectives

- Planner Agent orchestrates Weather + Attraction agents with real tool calling
- ChromaDB memory/RAG wired in
- Streaming AI responses to the chat UI

MVP Features: Real itinerary suggestions combining weather + attractions, conversation memory (agent remembers earlier messages in the trip) **Stretch:** basic conflict-resolution rule (e.g., "don't suggest outdoor attraction if heavy rain forecast")

Folder structure additions: `app/agents/planner.py`, `app/agents/weather_agent.py`, `app/agents/attraction_agent.py`, `app/tools/weather_tool.py`, `app/tools/places_tool.py`, `app/rag/chroma_store.py`

Tasks

- **A:** Build `weather_tool.py` (OpenWeather wrapper) and `places_tool.py` (Google Places wrapper) as clean, testable functions with typed responses
- **B:** Convert chat UI to handle streaming (SSE or WebSocket), show agent "thinking" states (e.g., "Checking weather...", "Finding attractions...")
- **C:** Build Planner Agent node that calls Weather Agent + Attraction Agent as tools via LangGraph, merge outputs into a structured itinerary draft; wire ChromaDB so past messages in a trip are embedded and retrieved as context
- **D:** Add `AgentRun` + `ItineraryDraft` tables/migrations to log every agent call (for later analytics/debugging); write integration tests for the full Planner → Weather/Attraction flow

Deliverable: User asks "Plan a 3-day trip to Goa in December," gets back a real, weather-aware, attraction-aware itinerary streamed into the chat.

Git workflow: same; tag this week's `develop` merge as `v0.1-agents-core` once demoed Friday.

Testing: Integration tests mocking external API responses (don't burn real API quota in CI); manual end-to-end test with real APIs before Friday demo.

Documentation: Document the agent state schema and message flow (sequence diagram) in `docs/architecture/`.

Submission checklist: Real itinerary generation  | Streaming chat  | Memory/RAG working 

Demo checklist: Ask for a real trip live, show streaming response, ask a follow-up question and show it remembers context.

Risks / buffer: External API rate limits during dev — Member A should add response caching (Redis) early so the whole team isn't burning the same quota testing.

Est. hours: ~16-18 hrs each (heaviest AI week).

WEEK 5 — Flights, Hotels, Budget & Dashboard

Objectives

- Complete the core agent set with Flight, Hotel, Budget agents
- Build the visual dashboard (not just chat) showing the itinerary

MVP Features: Flight search results, hotel search results, budget aggregation/breakdown, dashboard view of a trip **Stretch:** expense breakdown chart (pie/bar)

Folder structure additions: `app/agents/flight_agent.py`, `app/agents/hotel_agent.py`, `app/agents/budget_agent.py`, `app/tools/amadeus_tool.py`, `frontend/app/dashboard/`, `frontend/components/ItineraryCard.tsx`

Tasks

- **A:** Build `amadeus_tool.py` (flight + hotel search, sandbox), rate-limit + cache responses
- **B:** Build dashboard: trip overview card, itinerary timeline view, flight/hotel result cards, responsive layout pass
- **C:** Build Flight, Hotel, Budget agents; Budget agent aggregates flight+hotel+attraction cost estimates into a total with a simple breakdown; extend Planner to call all 5 core agents in sequence/parallel via LangGraph
- **D:** Schema additions for `FlightOption`, `HotelOption`, `BudgetBreakdown`; tests for Budget agent's aggregation math (this is the easiest agent to get subtly wrong — test it well)

Deliverable: Full itinerary with real flight options, hotel options, and a total budget breakdown, viewable in a proper dashboard (not just chat text).

Git workflow: merge to `develop`, tag `v0.2-full-mvp` after Friday demo.

Testing: Unit tests for budget math, integration test for full 5-agent pipeline, first round of frontend component tests (Jest + React Testing Library) for dashboard.

Documentation: Update API docs, add screenshots of dashboard to `docs/`.

Submission checklist: All 5 MVP agents working together ☒ | Dashboard live ☒ | Budget breakdown accurate ☒

Demo checklist: Full flow: chat request → dashboard shows flights, hotels, attractions, weather, total budget.

Risks / buffer: Amadeus sandbox data can be sparse for some routes — Member A should test with well-known routes (e.g., major Indian cities) for reliable demo data, and add a graceful "no results" fallback instead of crashing.

Est. hours: ~16-18 hrs each.

WEEK 6 — Feature Completion & Polish

Objectives

- Round out remaining planned features
- UI/UX polish pass — this is where "looks like a startup, not a college project" gets won or lost

MVP Features: Saved trips, trip history, maps integration (visual, not just data), packing list generator, dark mode, responsive polish **Stretch:** Transport agent, Food agent, in-app notifications

Folder structure additions: `frontend/app/history/`, `frontend/app/saved/`,
`frontend/components/MapView.tsx`, `frontend/components/PackingList.tsx`

Tasks

- **A:** `GET /trips` (history), `POST /trips/{id}/save`, packing-list generation logic (rule-based from weather + trip length + destination type — doesn't need to be a full agent)
- **B:** Trip history page, saved trips page, Google Maps embed with attraction/hotel markers, dark mode toggle (Tailwind `dark:` classes), full responsive pass on all pages, loading skeletons everywhere
- **C:** *If ahead of schedule:* Transport Agent and Food Agent (reuse Attraction Agent's pattern — same tool-calling shape, different Places API category filter). *If behind:* spend this week hardening the 5 core agents' prompt quality and error handling instead

- **D:** Notifications (stretch, simple in-app toast system, no need for push/email), full regression test pass, start writing E2E tests (Playwright) for the critical path: signup → plan trip → view dashboard → save trip

Deliverable: A feature-complete, good-looking app covering everything in the MVP list.

Git workflow: freeze new features after Wednesday this week — remaining days are polish/bugfix only. Tag `v0.9-feature-complete`.

Testing: E2E test for the critical user path, full regression across all previous weeks' features, cross-browser check.

Documentation: Update README with full feature list, start drafting user guide / demo script.

Submission checklist: All MVP features working ☒ | Responsive + dark mode ☒ | No known critical bugs ☒

Demo checklist: Full walkthrough rehearsal — time it, it should be under 5 minutes.

Risks / buffer: Feature creep is the biggest risk this week — if Transport/Food agents aren't clearly ahead of schedule by Tuesday, cut them without guilt.

Est. hours: ~14-16 hrs each.

WEEK 7 — Testing, Security & Admin Basics

Objectives

- Make it robust, not just working on the happy path
- Add minimal admin visibility (judges like seeing "we thought about ops")

MVP Features: Structured logging, global error handling, input validation/rate limiting, basic admin dashboard (usage stats) **Stretch:** simple analytics charts (trips created per day, popular destinations)

Folder structure additions: `app/core/logging.py`, `app/core/exceptions.py`,
`frontend/app/admin/`

Tasks

- **A:** Global exception handlers (consistent error JSON shape), Pydantic input validation audit on every endpoint, rate limiting via Redis (per-user, per-IP)
- **B:** Basic admin dashboard: total users, total trips, agent success/failure rate (pull from `AgentRun` table built in week 4)

- **C:** Prompt-injection and edge-case hardening pass on all agents (empty inputs, nonsense destinations, contradictory budget/dates) — write down 10 adversarial test prompts and fix what breaks
- **D:** Structured logging (JSON logs with request IDs), Sentry integration for error tracking, security audit: secrets not committed, `.env` in `.gitignore`, dependency vulnerability scan (`pip-audit`) / (`npm audit`) added to CI, load test the 5-agent pipeline with a basic concurrency test

Deliverable: App survives bad input, malformed requests, and concurrent load without crashing; admin can see basic system health.

Git workflow: bugfix branches only, tag `v0.95-hardened`.

Testing: Full test suite run with coverage report (aim 70%+ backend), security checklist signed off, load test results documented.

Documentation: Security notes doc, testing report, admin dashboard screenshots.

Submission checklist: Error handling audited ☒ | Logging + Sentry live ☒ | Admin dashboard ☒ | Security checklist ☒

Demo checklist: Deliberately send a bad request live to show graceful error handling — this is a strong demo moment, not a weakness to hide.

Risks / buffer: Don't let "security hardening" turn into a rabbit hole — timebox each item to a few hours; this is a student competition, not a bank.

Est. hours: ~12-14 hrs each.

WEEK 8 — Deployment, Docs & Demo Day

Objectives

- Ship it to the cloud
- Package documentation and rehearse the pitch

MVP Features: none new — deployment + docs only, no code changes after Wednesday

Stretch: none — this week is a hard feature freeze from day 1

Folder structure additions: finalize `.github/workflows/deploy.yml`, `docs/` fully populated

Tasks

- **A:** Finalize backend Dockerfile (multi-stage build), deploy to Render/Railway, environment variables configured in production, smoke test all endpoints against prod

- **B:** Deploy frontend to Vercel, connect to prod backend URL, final Lighthouse performance pass, fix any prod-only bugs (CORS, env var mismatches are the usual suspects)
- **C:** Final prompt tuning based on prod-like testing, write up the "how the multi-agent system works" section of the documentation with the architecture diagrams
- **D:** GitHub Actions CI/CD pipeline (auto-deploy on merge to `main`), final monitoring dashboard check, write final testing report, compile all documentation into a single submission package

Deliverable: Live, publicly accessible URL. This is the actual product.

Git workflow: `main` = production, every merge auto-deploys via Actions. Tag `v1.0.0`.

Testing: Full smoke test on production URL (not localhost) — this catches 90% of "works on my machine" surprises.

Documentation package:

- README (setup + architecture overview)
- SRS
- Architecture, sequence, ER, DFD diagrams
- API documentation (OpenAPI export)
- Testing report
- Deployment architecture doc
- Security notes
- Demo script (timed, <5 min)
- Slide deck for presentation

Submission checklist: Live URL working ☒ | All docs compiled ☒ | Demo script rehearsed by all 4 members ☒ | Slide deck ready ☒ | GitHub repo public/shared with judges ☒

Demo checklist: Full rehearsal at least twice as a team, each member should be able to explain any part of the system (judges ask random questions), have a backup screen-recording in case live demo/wifi fails.

Risks / buffer: Deployment surprises (CORS, env vars, cold starts on free-tier hosting) are almost guaranteed — this is why deployment starts Monday of week 8, not Thursday. Keep Thursday/Friday as pure buffer for exactly this.

Est. hours: ~10-12 hrs each.

6. Feature Scope Summary

Feature	Status	Week
Auth	MVP	3
Chat interface + streaming	MVP	3-4
Coordinator + Planner Agent	MVP	3-4
Weather Agent	MVP	4
Attraction Agent	MVP	4
RAG / conversation memory	MVP	4
Flight Agent	MVP	5
Hotel Agent	MVP	5
Budget Agent	MVP	5
Dashboard	MVP	5
Trip history / saved trips	MVP	6
Maps integration	MVP	6
Packing list	MVP	6
Dark mode / responsive	MVP	6
Transport Agent	Stretch	6
Food Agent	Stretch	6
Notifications	Stretch	6
Admin dashboard	MVP (basic)	7
Analytics charts	Stretch	7
Monitoring (Sentry)	MVP	7
Docker + CI/CD	MVP	throughout, finalized 8
Cloud deployment	MVP	8

When your team is ready to start any individual week, say "Start Week X" and I'll break it down into full implementation detail — code structure, actual code snippets, exact Git commands, DB migration SQL, agent prompts, and step-by-step deployment instructions for that week specifically.